1. Introduction

Reinforcement learning (RL) agents learn behavior by maximizing the reward defined by the designer. This creates a potential mismatch between the intended task and the measured objective: an agent can discover a strategy that produces high reward without accomplishing the task a person intended. This behavior is commonly called reward hacking.

This project studies reward hacking in a small, deterministic GridWorld. The research question is: how does reward-function design affect the behavior learned by an RL agent, and can a simple change, making a bonus reward collectible only once, reduce exploitation? The project compares a baseline goal reward, a repeatedly collectible bonus, and a one-time bonus.

2. Environment and MDP Design

The environment is a custom Gymnasium-compatible 8 by 4 GridWorld. The agent starts at (0, 0), the goal is at (7, 0), and the bonus location is at (4, 2). The layout, starting position, goal, bonus location, and 100-step episode limit are identical across conditions.

The state contains the agent's x coordinate, y coordinate, facing direction, and a binary variable indicating whether the bonus has been collected. Including the bonus flag makes the one-time-bonus environment Markov. The action space is discrete: turn left, turn right, or move forward. Transitions are deterministic. Reaching the goal terminates an episode; otherwise the episode is truncated after 100 steps.

3. Experimental Conditions

Only the reward logic changes between conditions. The exact values were locked before the baseline experiments so that the comparison remains interpretable.

Condition	Goal reward	Bonus reward	Purpose
Baseline	+10	0	Reference task behavior
Exploitable	+10	+1 on every bonus entry	Test repeated reward exploitation
Fixed bonus	+10	+1 on first entry only	Test a simple mitigation

The exploitable condition is intentionally designed so that the agent can receive repeated positive rewards by returning to the bonus location rather than completing the goal. The primary evidence of reward hacking will be a mismatch between high episode reward and low goal-completion rate.

4. DQN Method and Experimental Procedure

Agents are trained with the Stable-Baselines3 implementation of Deep Q-Networks (DQN) using an MLP policy. DQN estimates action values with a neural network and uses experience replay and a target network. The initial experiments used a discount factor of 0.99, a learning rate of 0.001, a replay buffer of 10,000 transitions, a batch size of 64, and the same exploration schedule across runs.

The milestone baseline uses 200,000 environment steps per run and 100 deterministic evaluation episodes. Four random seeds (0 through 3) were retained as separate runs. A hand-coded policy was tested first and reached the goal in seven actions, confirming that the task is reachable independent of learning.

5. Baseline Results

Table 1 reports the baseline DQN results. Seeds 0 and 1 learned the shortest seven-action path and reached the goal in every evaluation episode. Seeds 2 and 3 did not reach the goal and instead timed out after 100 steps. These results demonstrate that DQN can learn the environment, but performance is sensitive to random seed under the sparse goal reward.

Seed	Steps	Eval. episodes	Mean reward	Goal completion	Mean length	Training time (s)	Outcome
0	200,000	100	10.0	100%	7.0	47.2	Learned
1	200,000	100	10.0	100%	7.0	49.8	Learned
2	200,000	100	0.0	0%	100.0	46.1	Did not learn
3	200,000	100	0.0	0%	100.0	44.7	Did not learn

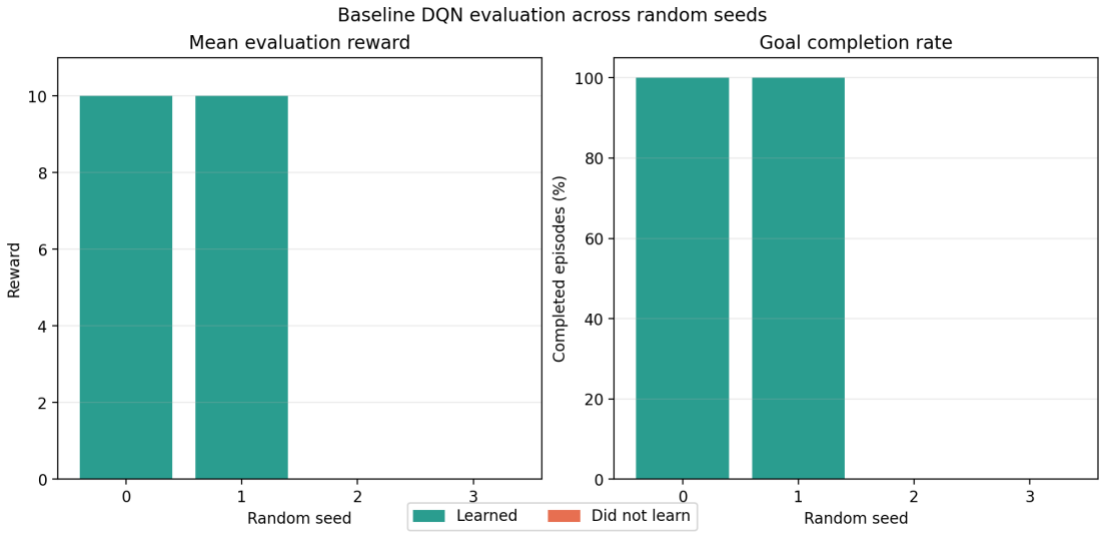


Figure 1. Baseline DQN evaluation results across random seeds. Green bars indicate runs that learned the task; red bars indicate runs that did not reach the goal.

The initial 50,000-step runs produced no successful baseline policies. Increasing training to 200,000 steps allowed some seeds to discover the goal path, identifying insufficient training for the sparse-reward task as an important factor. The unsuccessful seeds are not discarded; retaining them gives a more accurate picture of training variability.

6. Statistical Analysis Plan

The primary outcomes will be goal-completion rate, average episode reward, and reward-hacking rate. For the final comparison across reward conditions, each random seed will be treated as one independent experimental run. The 100 episodes generated by one model will be used to calculate that run's metrics, but will not be treated as 100 independent replicates.

For each condition, I will report the mean, standard deviation, and 95% confidence interval across seeds. If assumptions of approximate normality and equal variance are reasonable, I will use a one-way ANOVA followed by Holm-corrected pairwise comparisons. If those assumptions are not reasonable, I will use a Kruskal-Wallis test followed by corrected pairwise tests. Reward hacking will be defined as repeated bonus collection in an episode without reaching the goal.

7. Limitations and Next Steps

The current environment is intentionally small, deterministic, and fully observable. This makes behavior easier to interpret, but limits how broadly the results can be generalized. The turn-based action space and sparse goal reward also make DQN sensitive to exploration and seed. The milestone baseline therefore establishes that the implementation works and provides a useful learning benchmark, but it is not yet evidence that one reward condition is superior.

The next step is to run the same calibrated DQN procedure for all three reward conditions across multiple seeds, then evaluate reward, goal completion, episode length, bonus collections, and reward-hacking rate. Future extensions could include partial observations, randomized layouts, different bonus values, or a comparison with PPO.

8. Conclusion

This milestone established a working RL environment, locked the reward conditions, and produced baseline DQN results using Stable-Baselines3. The baseline agent learned successfully for some seeds, while other seeds failed under the same budget, highlighting the importance of reporting random-seed variability. The complete reward-hacking comparison will use this baseline as its reference condition and will evaluate whether limiting the bonus to one collection improves alignment with the intended goal.